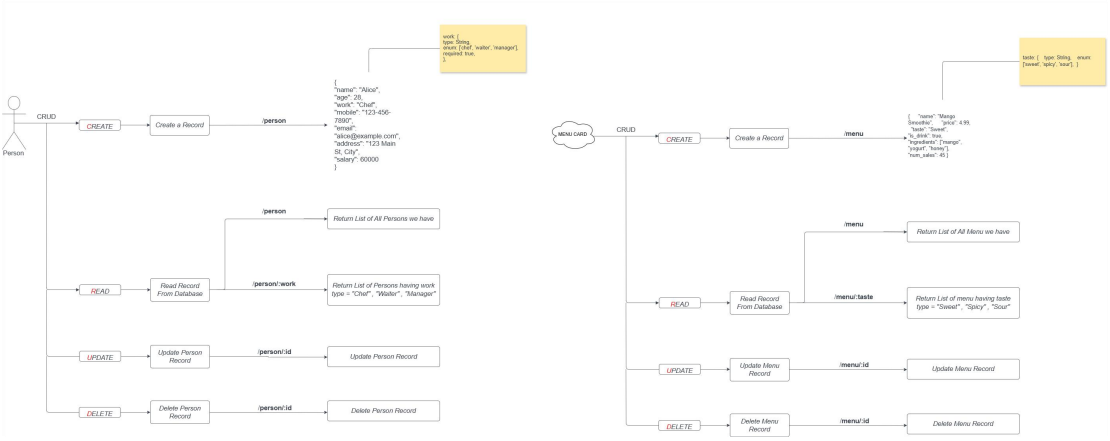
Week 10

Flow Diagram to API

An **API flow diagram** is a visual representation of the interaction between components in a system using APIs (Application Programming Interfaces). It depicts how data flows between different parts of an application, including clients, servers, databases, and third-party services.

Typically, an API flow diagram illustrates:

- **Endpoints**: The input and output interfaces of the API.
- **Requests**: The operations performed by the client (e.g., GET, POST, PUT, DELETE).
- **Responses**: The data or status returned by the server.
- **Actors**: Entities interacting with the API (e.g., users, systems, devices).
- **Sequence of Operations**: The step-by-step flow of calls and responses.



Parametrized API Calls

- Now if someone told you to give a list of people who are only ‘waiters’ or ‘chef’ or ‘manager’
- Then we can create endpoints like this:

  /person/chef
  /person/waiter
  /person/manager

➢ **But this is not the correct method to create as many functions here we can used parameterized endpoints**

- It can be dynamically inserted in the URL when making a request to the API.

- **Localhost:3000/person/:work**

→ **work = [‘waiter’, ‘manager’, ‘chef’]**

```
app.get('/person/:work', async (req, res) =>{
    try{
        const workType = req.params.work; // // Extract the work type from the URL parameter
        if(workType == 'waiter' || workType == 'manager' || workType == 'chef' ){

            //assumming you already have a Person Model and MongoDB connection set up
            const response = await Person.find({work: workType});

            if(response.length == 0)
            {
                return res.status(404).json({error: 'Data Not Found'});
            }
            console.log('response fetched');
            res.status(200).json(response);
        }else{
            res.status(404).json({error: 'Invalid Person work type'});
        }
    }catch(err){
        console.log(err);
        res.status(500).json({error: 'Internal Server Error'});
    }
})
```

Update Operation:

- We will update our PERSON record, for that we will create an endpoint from where we are able to update record
- For UPDATION we need two things:
    1. Which record we want to update ?
    2. What exactly we want to update ?

For update we will use ***PUT*** method to create endpoint

- ***What is a unique identifier in a document in a collection?***
    1. It's **_id** which is given by MongoDB itself, we will use this to find the articular record which we want to UPDATE
    2. And now we will send the data as same like we did in POST method

```
app.put('/:id', async (req, res)=>{
    try{
        const personId = req.params.id; // Extract the id of Person from the URL parameter

        const updatedPersonData = req.body; // Updated data for the Person

        //assume you have a Person model already
        const response = await Person.findByIdAndUpdate(personId, updatedPersonData, {
            new: true, // Return the updated document
            runValidators: true, // Run Mongoose validation
        })

        if (!response) {
            return res.status(404).json({ error: 'Person not found' });
        }

        console.log('data updated');
        res.status(200).json(response);
    }catch(err){
        console.log(err);
        res.status(500).json({error: 'Internal Server Error'});
    }
})
```

Delete Operation:

- We will DELETE our PERSON record, for that we will create an endpoint from where we are able to DELETE record
- For DELETION we need one thing
  1. Which record we want to Delete?
- ***What is a unique identifier in a document in a collection?***
  1. It's **_id** which is given by MongoDB itself, we will use this to find the articular record which we want to DELETE

```
app.delete('person/:id', async (req, res) => {
    try{
        const personId = req.params.id; // Extract the Person's ID from the URL parameter

        // Assuming you have a Person model
        const response = await Person.findByIdAndRemove(personId);

        //If no Person found
        if (!response) {
            return res.status(404).json({ error: 'Person not found' });
        }
        console.log('data delete');
        res.status(200).json({message: 'Person Deleted Successfully'});
    }catch(err){
        console.log(err);
        res.status(500).json({error: 'Internal Server Error'});
    }
})
```

Express Router:

- We have lot of endpoints in index.js file (Server File)
- This makes bad experience in code readability as well as code handling
- Express Router is a way to modularize and organize your route handling code in an Express.js application
- So, let's create a separate file to manage endpoints **'/person'**
- Express Router is like a traffic cop for your web server
- Express Router helps you organize and manage these endpoints in your web application. It's like creating separate folders for different types of tasks.

- Create a folder **routes** ⟶ **personRoutes.js**

JS personRoutes.js ●

routes > JS personRoutes.js > ...

```
1   const express = require('express')
2   const router = express.Router()
3   const Person = require('./../models/Person')
4
5   router.post('/', async (req, res) =>{
6       //Handle POST '/person' Here
7   })
8
9   router.get('/', async (req, res) =>{
10      //Handle GET '/person' Here
11  })
12
13  router.put('/:id', async (req, res) =>{
14      //Handle PUT '/person/id' Here
15  })
16
17  router.delete('/:id', async (req, res) =>{
18      //Handle DELETE '/person/id' Here
19  })
20
21  module.exports = router;
22
```

- Now in **index.js (Server File)** we will use this **personRoutes.**

```
//import the router files
const personRoutes = require('./routes/personRoutes')

//use the routers
app.use('/person',personRoutes);
```